Python

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

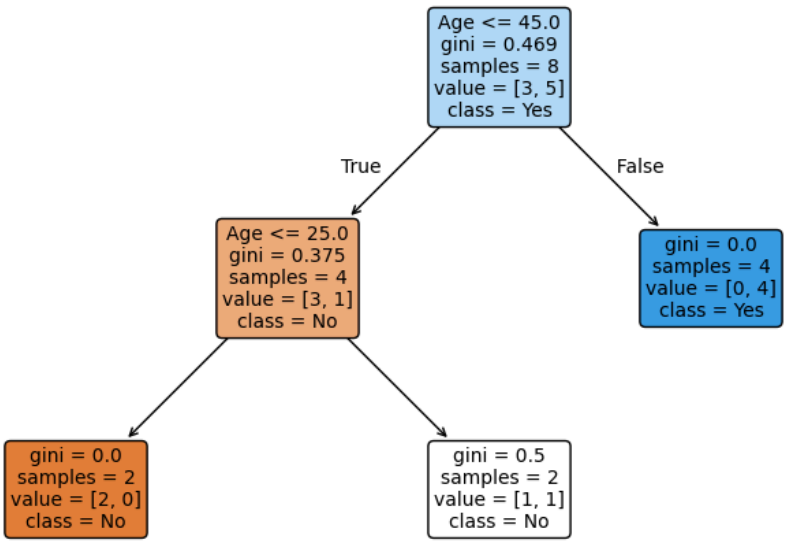
# 1. Dataset with clean, easy numbers (10, 20, 30...)
df = pd.DataFrame({
    "Age": [10, 20, 30, 40, 50, 60, 70, 80],
    "Salary": [10, 20, 30, 40, 50, 60, 70, 80],
    "Purchased": [0, 0, 1, 0, 1, 1, 1, 1]
})

X = df[["Age", "Salary"]]
y = df["Purchased"]

# 2. Train Model (max_depth=3 for bigger tree)
model = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X, y)

# 3. Canvas Size Bada aur Font Compact (No Overlap)
plt.figure(figsize=(14, 7), dpi=150)
plot_tree(
    model,
    feature_names=["Age", "Salary"],
    class_names=["No", "Yes"],
    filled=True,
    rounded=True,
    fontsize=8,
    impurity=False
)

plt.tight_layout()
plt.show()
```



```python
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier

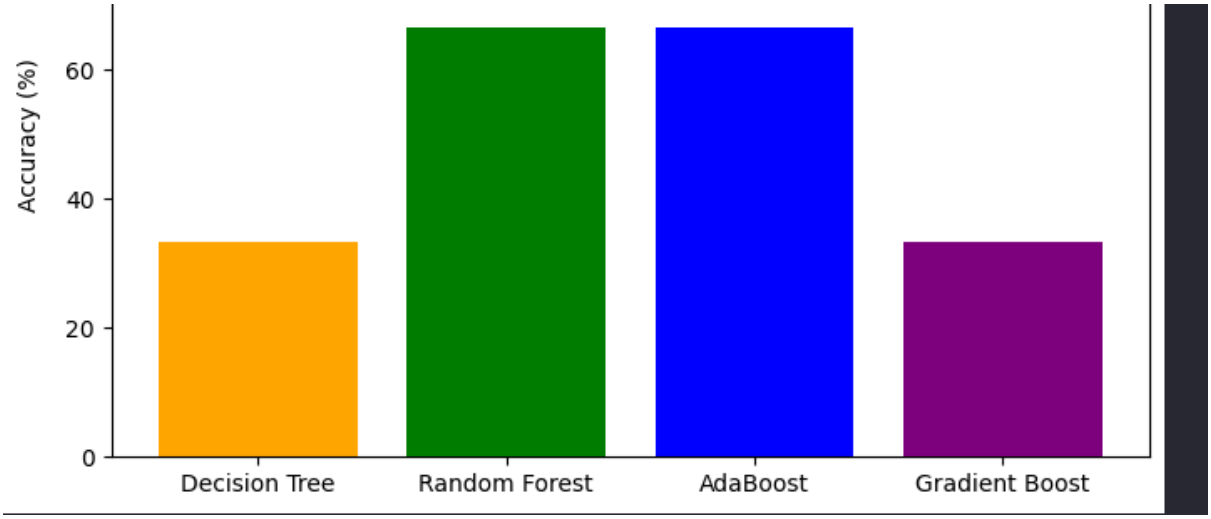
# 1. Clean yet Realistic Dataset (Thoda overlap daala hai realistic output ke liye)
X = [[10, 20], [20, 30], [30, 25], [40, 50], [50, 40], [60, 70], [70, 60], [80, 90], [90, 85], [100, 95]]
y = [0, 0, 1, 0, 1, 1, 0, 1, 1, 1]

# 2. Split Data
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

# 3. Models Setup
models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=10, random_state=42),
    "AdaBoost": AdaBoostClassifier(n_estimators=10, random_state=42, algorithm='SAMME'),
    "Gradient Boost": GradientBoostingClassifier(n_estimators=10, random_state=42)
}

# 4. Train & Evaluate
acc = {}
for name, m in models.items():
    m.fit(X_tr, y_tr)
    acc[name] = round(m.score(X_te, y_te) * 100, 2)
    print(f"{name}: {acc[name]}%")

# 5. Plot Realistic Comparison
plt.figure(figsize=(8, 5))
plt.bar(acc.keys(), acc.values(), color=['orange', 'green', 'blue', 'purple'])
plt.ylabel("Accuracy (%)")
plt.title("Ensemble Models Comparison (Realistic)")
plt.ylim(0, 100)
plt.show()
```



```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.metrics import ConfusionMatrixDisplay

# 1. Dataset (Clean 10, 20... values)
X = np.array([[10,20], [20,30], [30,25], [40,50], [50,40], [60,70], [70,60], [80,90]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

# 2. Train Model & Print Accuracy
model = SVC(kernel="linear").fit(X, y)
print("Accuracy:", model.score(X, y) * 100, "%")

# 3. Plot 1: Confusion Matrix
ConfusionMatrixDisplay.from_estimator(model, X, y, display_labels=["Apple", "Orange"])
plt.show()

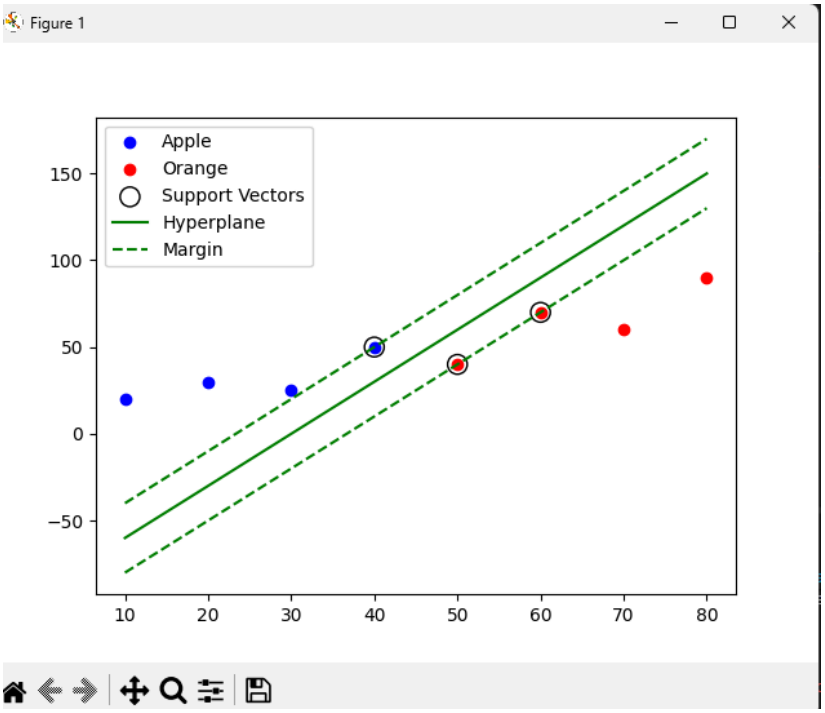
# 4. Plot 2: Scatter Data, Support Vectors, Hyperplane & Margins (With Legend)
plt.scatter(X[y==0, 0], X[y==0, 1], c="blue", label="Apple")
plt.scatter(X[y==1, 0], X[y==1, 1], c="red", label="Orange")
plt.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1], s=120, facecolors="none", edgecolors="k", label="Support Vectors")

# Line Math (Hyperplane & Margins)
w, b = model.coef_[0], model.intercept_[0]
x_line = np.array([10, 80])

plt.plot(x_line, -(w[0]*x_line + b)/w[1], "g-", label="Hyperplane")
plt.plot(x_line, -(w[0]*x_line + b - 1)/w[1], "g--", label="Margin")
plt.plot(x_line, -(w[0]*x_line + b + 1)/w[1], "g--")

plt.legend()
plt.show()
```

Loading...



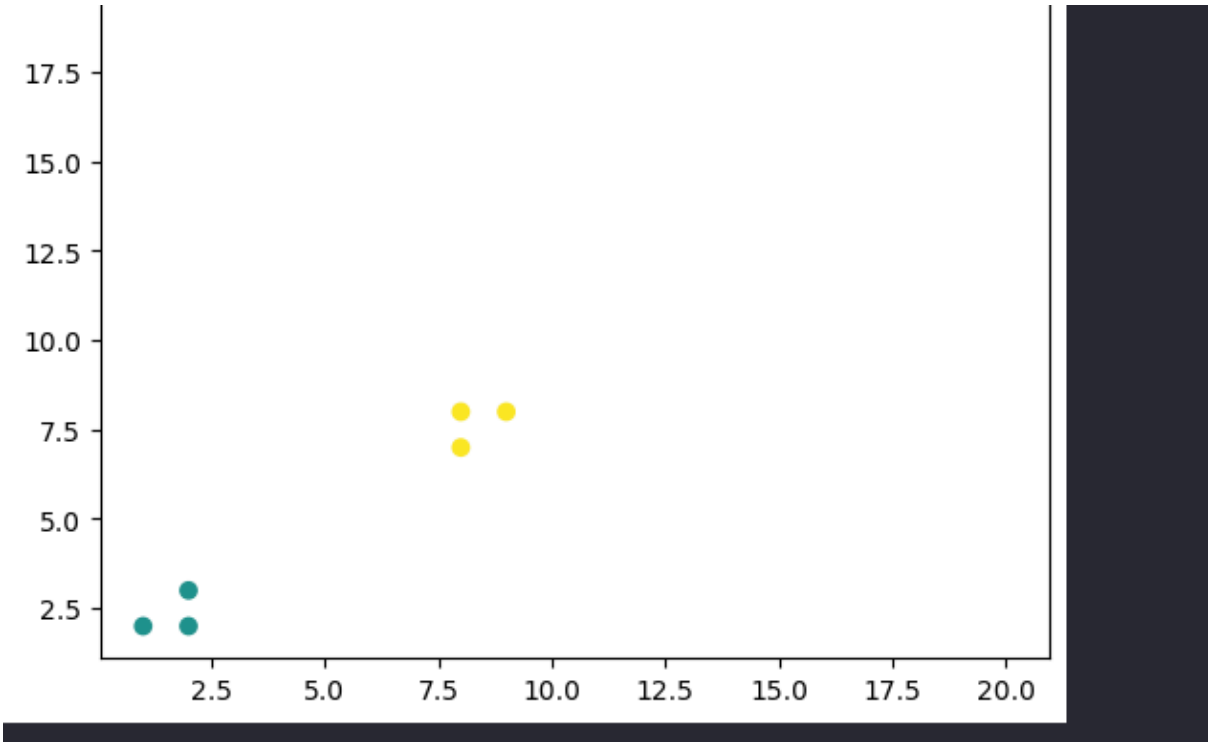
Python

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN

# 1. Dataset (Clean array)
X = np.array([[1, 2], [2, 2], [2, 3], [8, 7], [8, 8], [9, 8], [20, 20]])

# 2. Model Train & Labels Print
labels = DBSCAN(eps=2, min_samples=2).fit_predict(X)
print("Cluster Labels:", labels)

# 3. Plot Clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis")
plt.title("DBSCAN Clustering")
plt.show()
```



```python
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Simple User Input (Space se separate karke daalo)
print("Enter 3 values per row (e.g. 10 20 30):")
row1 = [float(x) for x in input("Row 1: ").split()]
row2 = [float(x) for x in input("Row 2: ").split()]
row3 = [float(x) for x in input("Row 3: ").split()]

X = np.array([row1, row2, row3])

# 2. Scale & Apply PCA
X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 3. Print Results
print("Transformed Data:\n", X_pca)
print("Variance Ratio:", pca.explained_variance_ratio_)
print("Total Variance Retained:", sum(pca.explained_variance_ratio_))
print("Components:\n", pca.components_)
```

```
PS C:\Users\Eesha Potnis\Desktop\ML-project> python exp.py
Enter 3 values per row (e.g. 10 20 30):
Row 1: 10 20 30
Row 2: 12 13 14
Row 3: 34 35 36
Transformed Data:
 [[-0.42746623  0.81603288]
 [-1.74731878 -0.54142437]
 [ 2.17478501 -0.27460851]]
Variance Ratio: [0.88506002 0.11493998]
Total Variance Retained: 0.9999999999999999
Components:
 [[ 0.56229727  0.6135684   0.55439661]
 [-0.68222501 -0.03467178  0.73031972]]
PS C:\Users\Eesha Potnis\Desktop\ML-project>
```